Chapter 5

Directories and Naming

A *directory* is an object that maps names to descriptions, identifiers or references. It is also logically a distributed object with shared state that needs to be kept consistent, namely the state that maps names to these descriptors or references.

In the general object-oriented structure of systems, the (global) directory maps names to object references, which then allow operations on this object. For example,

```
String name = . . .;
// Lookup object in directory by ``name''
Ptr<NamedInterface> o = Directory::root()->interface( name );

// Cast pointer to a file from a raw pointer if actually a file pointer.
Ptr<OpenFile> f = dynamic_cast<OpenFile *> o->rawPointer();
if( f == 0 ) { // Not a file
   return an error
  }
f->read( buf, bytes );
```

where `Directory::root()` is the root directory. Here, the directory returns a reference to a file object corresponding to the specified name as a pointer to a top-level type, `NamedInterface`. This type is then downcast to the expected `OpenFile` type to access the type-specific interface to invoke the operation. Then, it invokes the read call on this open file object, using the OO dispatch of mapping to the object and then to the operation associated with this object.

Note: the "directory" here might actually be viewed as a directory system in the sense that normally a name is a hierarchical path name and the lookup is performed using several directory objects, one for each component in the path name. Also, a directory lookup is returning a reference to an interface, not the object. For

example, the data in a file may be accessed by several different interfaces. A name lookup returns a reference to one particular interface instantiation.

A unified system-wide directory allows names to represented, transmitted and used across the system in the same way, independent of location, unambiguously. Otherwise, the same name may have a different meaning or designation on different nodes and different types of objects may be named by different name types.

You might ask: Why names? There are three key reasons, of course:

1. **Correctness**: (versus use of descriptions) A (proper) name identifies a specific entity unambiguously whereas there may be multiple entities matching a description. Moreover, multiple copies of descriptions lead to inconsistencies when object is modified, e.g. "man with the green hat" — suppose he takes off the hat?

2. **Efficiency**: names are shorter, easier to manipulate than entities named. E.g., short names of objects or descriptions rather than the entities themselves.

3. **Flexibility**: provides a level of indirection, symbolic reference, allowing rebinding specifics and details. e.g. "VP of engineering" vs. the specific person when you want to designate the individual in the role at a given time, not a particular individual. Adage: *Every problem in computing can be solved by an extra level of indirection, except performance.*

More broadly, the universe consists of trillions of objects spread across numerous individual computing nodes and distributed (sub)systems. You would like to be able to identify each of these objects in time and space (i.e. logical object and object version) given there are no clear boundaries between systems with everything being Internet-connected. Thus, the (distributed) directory object is a key issue for what Einstein might have called the 4-D time-space computing continuum, with scaling being a critical requirement.

Directories and naming have been an important component of several previous objects, including distributed shared memory, clocks, and of course, file systems. Naming is in fact one of the key functionalities of truly large-scale distributed systems.

This chapter explores the challenge of building a truly scalable naming architecture and supporting directories in a distributed system. We first explore what one might call naming theory, including some useful terminology, providing insight into naming semantics, the basis for providing and enforcing sufficient consistency.

5.1 Naming 'Theory'

A *name* is a *value* used to refer to or identify an object or collection of objects.

Note: a *value* is generally defined by equality. That is, there is some notion of equality when two values are "the same". This contrasts with objects such as files. For example, /usr/bin/emacs and /usr/local/bin/emacs may have same contents (or value) but different files.

Opinion: a name is a name by its use, not some intrinsic property.

Philosophical problem: must a value refer to an existing object to be a name? e.g., 'President of Canada', 'King of France', 'the P=NP proof'. If yes (as per Russell), then undecidable!

Name interpretation: deciding which object or collection of objects a name identifies, i.e., what does a name *mean* or refer to.

Naming: binding, mapping and listing of names.

Naming Terminology An name x is *bound* to an entity y, the thing it names, i.e. it's *binding*.

That is, y may be the *definition* of an object, rather than the object itself, introducing a level of indirection.

For example, a file descriptor record *defines* or describes the file but is not the file itself.

A binding is represented as a naming tuple (x,y), created by this *binding*.

A *context* is a set of naming tuples. (x,y) in context C means x names y in context C.

A name x is *unique* in a context C if only one name tuple with first element x.

A name is *invalid* if there is no naming tuple in C with first component x.

A name is *ambiguous* if there are multiple tuples with first element x in C. The term *generic* is also used, but has the connotation of referring to multiple instances of a common types, often distinguished or disambiguated by more local context. E.g. "/dev/tty" in Unix.

A name x is an *alias* for u in C if both (u,y) and (x,y) occur in C.

A *name mapping* for C is a function that, given x, returns y such that (x,y) is in C. If x is ambiguous in C, it may return all such y or a random one. Composite name mappings possible if y is also a name, presumably in another context.

Note: name is bound once (normally) but may be mapped numerous times.

5.1.1 Proper Names, Common Names and Descriptions

Considerable work in logic, philosophy and mathematics on naming.

> *Logicians tie themselves in knots, e.g., 'I can juggle three balls.' and '3 is necessarily less than 5' implies (?) 'I can juggle necessarily less than 5 balls, by substitution.'*

Important distinction:

- reference — denotation — extension.
- sense — connotation — meaning.

A name is a *proper name* if it denotes but lacks connotation or meaning. Entity is 'baptized' with name, paradigm 'Socrates'. A proper name is like a variable name or instance name in programming. The *extension* or *extent* is the range over which a variable of this type can match, similar to a key of a relation in the relational database context.

A *description* is a set of properties that an entity satisfies, which may be satisfied by zero, one or more entities.

A *definite description* is one that describes only one entity. It may be regarded as equivalent in this sense to a proper name. However, a description can lose its definiteness by other entities coming into existence or changing to match this description or the described entity changing.

A *common name* is a symbolic reference to a definition or description. A type name in a programming language is an example of a common name.

All entities satisfying the definition are so named without 'baptism'.

Also, unlike proper names, common names have properties of:

- synonymy — multiple names for same object.
- ambiguity — name multiple objects, namely all objects matching the description.
- substitutability — replace common name with description. e.g., 'That is a chair' or 'That is a piece of furniture for sitting, etc.'
- redundancy — description may entail more than required to identify, e.g. "with typically 4 legs, though sometimes 3."

A *dictionary* contains the descriptions (and synonyms) for common names.

Logicians (Katz 1977) distinguish between:

- type reference — referents of common name in general, e.g., "a chair is needed".

- token reference — referent in a particular use. E.g., 'Sit in the chair.' referring to a specific chair.

Computer example: '/dev/tty' is common name for *your* terminal in Unix, which refers to specific terminal on use.

Names are used for communication: sender or user of name plus receiver or interpreter of name.

Thus, one can distinguish between the *intended referent* versus *actual referent*. Ideally, these are the same entity, of course.

Overall, names play a strong role as an optimization over using descriptions. In that vein, it is interesting to note that dictionary-based compression works on the same principle, i.e. use a reference to a dictionary entry rather than sending the "value" in the dictionary.

5.1.2 Names versus Types

Name — an object instance's *proper name*.

Type — an object instance's *common name*, i.e. the class, descriptor to which matches. An object can have multiple common names, informally or formally (e.g. inheritance).

Some systems provide this category of information as part of the directory system, including:

- representation
- protocols with which to access object.
- operations it supports

For example, interface repository of CORBA provides some of this type of support. However, this information is normally only useful for operations dealing with collections of objects that can be of different type, such as a directory listing. In normal operation, the client knows the expected type or types and simply attempts to downcast to the expected type to determine whether the object is or is not of that type.

The Unix file `stat` operation provides some file information which is logically part of the directory, but not implemented that way.

Names are normally well-known types, either character string names or fixed-length bitstrings (in the case of internal names.)

5.1.3 Pure Names and Impure Names

A *pure name* is one that contains no encoding of a description of an entity. For example, "chair" does not contain any description of a chair object.

An *impure name* includes some description of entity named. For example, the name 'bicycle' indicates 'has two wheels', if you know Greek. More blatantly, "William of Warwick" indicates where William lives, or at least where he is from (or used to be from).

Advantages of impure names:

- can encode information to facilitate name mapping, e.g., telephone numbers — encode area and exchange.
- can provide holder of name with information about entity named without having to access object description, e.g., '/dev/tty' indicates its a device (by Unix convention).
- Facilitates distributed allocation because of encoded info. For example, the phone company can allocate a new phone number in the local exchange without danger of conflict with the rest of the world because the area code and exchange are encoded in the number. That is, they are really just allocating the last few digits of the global number which only have to be unique local to or within the local exchange.

Disadvantages are size of name and restrictions on changes to entity without changing name. For example, you cannot move to a different area without changing telephone number, in particular the area code of your phone number.

Note: there is a spectrum between *pure names* to *pure descriptions*, with intermediate examples that are names but contain some degree of descriptive information.

5.1.4 Structured Names

Structured name: composed of one or more component names, e.g., hierarchical file naming, telephone numbers, Internet host addresses. The structuring of names is really a property of a context or type of name, rather than an individual name. For example, it only makes sense to talk about `/dev/tty/ttya` as structured within the context of Unix device names. In other contexts, its structure might be meaningless, in particular in a context in which "/" is not a delimiter.

Structured names can be pure but normally structure is a convention for encoding descriptive information and hierarchical context. For example, "/" is the root context, "/dev" is a subdirectory context, and so on, each localizing the name allocation problem to the individual directory.

In this vein, scalable naming is *hierarchical*, where each name is structured as a sequence of component names, each component name only being required to be unique relative to the context specified by its prefix components, and there is a top-level root context. For example, the component name "md" in the hierarchical (structured) name "/dev/md/config" is only required to be unique relative to the context specified by "/dev". Here, "/" specifies the top-level root context.

The term *name transparency* is refers to being able to move a named object without changing its name. The term *location transparency* refers to being able to use name in different places and have it mean the same.

5.2 Naming and References in Object-oriented Systems

In a truly object-oriented system, there is uniform identification of objects independent of type. Ideally, there are character-string names of objects as well as fixed-length bitstring identifiers to use as efficient object references, both applicable to all types of objects. The remote object system typically recognizes object references that are passed so it can perform the required transformations between machines.

Naming operations include:

1. create an object with a new name or, relatedly, create a new (alias) name for an existing object.
2. map a given name to the object it names.
3. invalidate a name. One can also view deletion from the name space as changing the mapping of the name to the null object. Note: deleting a name from a directory does not generally result in deleting the object unless there are no outstanding references to this object. I.e. the behavior should be the same as with Unix files: a file that is deleted from a directory is not actually removed until the last open file for this file is closed (i.e. the last reference is removed).

Also, directory operations to list names within a context, i.e. iterate over the entries in a directory.

In contrast to object-oriented designed systems, older systems have different name spaces, identifier formats and contexts for different types of objects. For example, in original Unix [1], files are identified by character-string names and file descriptor identifiers that are local to the address space. In contrast, processes only had a single

[1] More recently, Unix and Linux support a "/proc" directory that incorporates processes into the file name space.

system-wide process identifiers. Moreover, object references are passed as ordinary or non-distinguished parameters in calls, making automatic reference management as part of marshal/demarshal impossible.

5.2.1 External versus Internal Object Names

Objects generally have both an *external name*, chosen by the user as well as a *internal name* or object references/ids, chosen by the system. Let's consider why.

Users need to have user-meaningful or external names for:

1. referring to objects from the command line, scripts and GUI menus.
2. meaningful error reporting
3. address-space and machine-independent identification of objects, including passing identification of objects between machines.

Absence of this naming support means that applications often develop *ad hoc* naming for specific classes of objects, resulting in duplicated and inconsistent mechanisms. The naming is also not system-wide, causing confusion in the error reporting and logging and making it infeasible to use for point 3. above. By "system-wide", I mean that a name can refer to the same interface anywhere in the system, across multiple nodes in the distributed case, which is what we are concerned with.

System and application programs are more efficient, simpler to implement and more robust using short, fixed-length, bit-string names, an *internal name*. e.g., integers or pointers to identify open files. Usually generated by the system module that implements the corresponding object. E.g. fileIds are allocated by the file system module.

Thus, *External names are a necessity; internal names are an optimization.*

5.2.2 Conventional Naming Thinking vs. Cheriton's Approach

Conventional thinking calls for a centralized *name server* that maps external names to some internal identifier, along the lines of DNS, for example. The client uses this server to get the internal identifier for an object and the server on which it resides, and then accesses the object on this server specifying the internal identifier, usually a UUID (univerally unique identifier). However, this approach requires the internal identifier have the semantics of a full proper name for the object.

The alternative view I favor is one in which each object module implements the external names for the objects that it implements. That is, if object "foo" is implemented by some instance IM of module M, then IM contains the directory mapping

"foo" to this object. This is the same approach used by file systems, where the file data and metadata plus file directories are all contained on the same volume, standard practice for the last 30 years or more. This makes sense because a module must supply names or references for the objects it implements, either internal or external names (i.e., bit-string or character-string names). Otherwise, how do you specify the object to instantiate when contacting the module?

In particular, consider if disk file is named with UUID bit-string (such as Cambridge Universal file server[5]). The file server must implement a directory mapping a UUID to the file object when an open file object is created. Thus, some level of naming must be implemented in each object module. It's purely an issue of whether this directory maps names or UUIDs.

In general, modules that use such internal names in their interface must rely on an additional (separate) directory mechanism that provides external names for these objects. This introduces the problem that the external name directory service may be unavailable, rendering the objects inaccessible to the client, even when the object module is available. This structure violates the *fate sharing principle* of networking and distributed systems, where facilities required to provide a service should designed to be either up together or down together.

With external naming per module, a separate (distributed) name or directory server only acts to map a name provided by a client to the object module that implements the named object. Thus, it implements name or name-prefix to server name, and server name to network identifier.

Applied recursively, this approach means a network directory should then map a server host name to the host that "implements" this name. By the same decentralized naming argument, routers should implement this level of naming. This is in fact the name-based routing approach advocated by Gritter and Cheriton, recognizing that the conventional routing table can be extended to map name to routes, rather than just addresses to routes.

Thus, the ideal is for external naming to be distributed across object modules, with each object module (logically) implementing the external names for the objects it implements.

The next two sections explore internal naming and external naming further, pointing out the common mistakes and supporting the Cheriton approach, of course.

Note: The common problem with systems design is overburdening internal names because of deficiencies in the external naming support. This is another instance of putting too much at too low a level.

5.3 Internal Naming

An *internal name* is one that is generated by the system or application (rather than selected by the user), usually designed to improve the efficiency of mapping to the referenced object. Thus, they are typically (small) fixed-length bitstrings of a power of two length (friendly to computer memory/register units) often with some structure to facilitate mapping. Typical examples include: memory addresses, file descriptors, process identifiers and connection identifiers.

5.3.1 Case Study: IP Addresses

An *IP address* is an internal address used to identify a host interface to the Internet. That is, it does not identify a host, but a communication interface of an Internet host. The IPv4 address is 32 bits, an efficient value for 32-bit computers. IPv6 addresses are 128 bits long, less efficient but the same basic idea.

In original specification, IPv4 addresses were structured with a network number component which varied between different classes of addresses. Class A addresses used 8 bits of network number, class B used 16, and class C used 24 bits, leaving 24, 16 and 8 bits for the host component respectively for each class of address. This *impurity* in the address means a host cannot move between networks without changing its address.

In the early days of the Internet, these classes allowed an efficient mapping of IP addresses. For instance, Stanford had a class A address of net 36, so any packet addressed to 36.X.X.X could simply be forwarded "towards" Stanford by routers world wide. However, the size of the classes turned out to be a bit awkward as the Internet grew. There were very few class A addresses and the holders of same were not using them very efficiently. For example, Stanford had around 30 thousand hosts, nowhere near 16 million (at the time it gave up its class A address). On the other hand, with the proliferation of cheap PCs, a class C address of 254 hosts (removing two for broadcast addresses) was frequently insufficient[2].

This problem is a common one with fixed structure of internal names: the choice of structure often does not work ideally with the use as the use grows. The introduction of *CIDR*(*Classless InterDomain Routing*) has provided far more flexibility in the allocation of IP addresses, staving off IPv4 exhaustion for a time.

Another deficiency with IP addresses was their manual configuration in host machines, especially for large-scale dynamic networks. Each host has to be manually configured or reconfigured, creating a significant management overhead. DHCP

[2]Given these problems with addressing structure, it is surprising that IPv6 is proposing to go a similar route with a whole *addressing architecture* that specifies a number of subfields. I cannot identify any addressing structure or addressing architecture that has been a technical success.

(Dynamic Host Configuration Protocol) was developed to allow automatic dynamic allocation of IP addresses to hosts. The assignment may depend on the external name of the host or its MAC address, provided in the DHCP request from the host.

With hosts that move around, such as laptops, this dynamic assignment allows a host (interface) to be assigned an address in its current location that is within a pool of addresses used by that location already, allowing the infrastructure to be simpler and more stable than trying to accommodate foreign addresses. For example, when I am at a conference, my laptop may be assigned a 10. private address by the wireless access point whereas at Stanford, my laptop might be assigned an address from the 171. block belonging to Stanford.

However, the net result of the DHCP improvement is that IPv4 addresses are transient routing tags or *hints* on how to get a packet to a particular host interface. There is no guarantee on how long this internal identifier continues to provide routing to this interface and how long after it ceases to do that it may provide routing to another Internet interface. I.e. you cannot trust an IP address to mean something over time.

In addition to dynamic allocation, the use of *network address translation*(*NAT*), typically by a firewall separating an enterprise network from the Internet, allows the IP address assignment to be just locally unique as well as dynamic. NAT relies on external (DNS) naming to really identify an interface, rendering the IP address far less meaningful. NAT is basically like virtual memory: it provides a local identifier space that can be managed locally and independently of a global space.

Thus, the Internet has evolved to use IPv4 addresses as transient internal names for host interfaces without global meaning, relying more and more on DNS (external) names. In this vein, IPv6 is far less compelling with this direction because it is primarily motivated by the view of making addresses first-class names of hosts, not reusable transient routing tags.

There has been work proposing to separate location and identification as part of network-level addressing. e.g. LISP (Location/ID Separation Protocol by D. Farinocci et al.). However, this work, IMHO, fails to recognize that an IP address has evolved to just a transient routing hint and that external names are the only reasonable means of identification. (Lots of this confusion comes from the other view that one wants to maintain the same TCP connection during mobility. In reality, you need to switch from one connection to another, relying on a session layer to migrate the application from one connection to another.) I think of this like Tarzan: you achieve mobility by being able to smoothly swing from one vine to another, not by trying to move one vine between trees. TCP connections are like vines; you want to firmly establish a new connection during mobility before letting go of the current connection.

5.3.2 Case Study: V Process and Object Identifiers

The V process identifiers were originally structured as *logical host* subfield followed by local identifier. Notion: logical host identifies host, local identifier assigned by

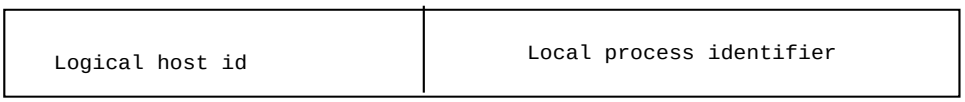

Figure 5.1: Old V Process Identifier Structure

host. Note: for transparent IPC, the collection of kernels running on nodes in some "system" try to act as one module for purposes of naming.

Advantages:

1. Assists process id mapping — one kernel entry per logical host, not per process.
2. easy distributed unique pid generation.
3. easy invalidation.

Extension for migration: multiple logical hosts per physical host.

Similar to the *file groups* used in Unix — file group is the unit of migration, replication, etc.

Conclusion: a design mistake with migration.

Right solution: allocation blocks of pid's: replace logical host with allocation block id.

One cache entry per pid is not too bad, primarily just for server processes.

Problem: testing for validity of pid in cluster of machines.

Solution: remember recently 'deceased' pid's avoids probing the network most of the time.

T-stability: a process identifier is not rebound from one process to another in less than time T.

An object in V is designed as

```
( server-id, local-object-id)
```

where the `server-id` was the process identifier of the process executing the service and the `local-object-id` was a value unique to that service. This object identification is system-wide unique in V because the `server-id` is, and the server ensures

that the local identifiers are unique locally. That is, `server-id` effectively specifies a context for `local-object-id`.

The V approach illustrates one approach to object identifiers: make them system-wide unique by relying on some unique identifier supplied by the communication system. As discussed above, this has the problem of requiring rather large identifiers. However, only the local object identifier needs to be sent in the common case of communicating it to the server, because the server-id is implicitly based on who it is sent to. In V, these identifiers were for transient objects only, with character string names for persistent objects. These identifiers also tied the object to a particular server process, which precluded transparent object migration.

A better approach is to select an object identifier as local to the context of a communication channel from client to server (rather than independent of client in V.) That is, a communication channel is a context for internal (local-to-connection) identifiers. For instance, an object id used over a TCP connection need only be unique and useful within this connection. Note: it is rarely practical to use a communication channel per object, so some form of local identifier in the channel is required in the general case. With the communication channel providing a context for this identifiers, the two endpoints can know when this connection/context is terminated, so identifiers only need to be made unique over the time period in which the connection is valid, and only relative to the connection. In this vein, it is important to firmly have in mind that a connection is just synchronized state between endpoints, not really a "connection" in the physical sense.

5.3.3 UUIDs Considered Harmful

A number of systems have been developed with *universally unique identifiers*(*UUID*)s. These are basically internal names that are burdened with being persistently unique in space and time across the "universe".

This approach was used by the Apollo DOMAIN system as well as a number of academic systems, but they are a bad idea that keeps coming back with each generation of "innovators". DOMAIN used large persistent object identifiers that encoded the node on which the object was created to aid in mapping the UUID to the object, which was basically/primarily a file. DOMAIN then supported forwarding of references to new nodes if the object was migrated.

However, this approach has a number of significant problems. First, how big should an internal identifier be?

For it to be unique in space and time, it needs to accommodate $R*Tinfinite$ where R is the expected rate of object creation. This typically leads to estimates based on objects being created every picosecond for 200 billion years. So, perhaps 128 or 256 bits for a UUID — I didnt do the math.

Now, with this large identifier, it can be very expensive to allocate and map, so many designs introduce "impurities" to facilitate mapping, e.g., embed IP address to ease mapping[3]. Impurities increase the required size even further. The size now starts to get in the way of efficiency, the real reason for internal names. Moreover, most uses of identifiers are very local, and do not need the identifier to be globally unique, often spawning additional layers of internal names/identifiers, or variable-length UUIDs.

The use of UUIDs means that a client typically has to access a directory to map the (external) name to a UUID as part of accessing the object, introducing more overhead. Moreover, there is the additional problem of accessing objects when the directory service fails. A similar issue arises with security. The client needs to trust/authenticate the directory system to return the correct UUIDs and then trust/authenticate the object module that implements this UUID. Overall, this approach causes a dependence on a third party that is not otherwise required for the interaction, i.e. the name-to-UUID server.

This dependence on a 3rd party entity or what networking folks call an "off-path" dependency is far worse than you might imagine, considering high availability and security. In particular, there are varying degrees of availability and reliability required for different services. A server implementing a service can be designed and configured/deployed to match these requirements. However, a 3rd party that the service must depend on then needs to be designed and configured to match these requirements. Specifically, it generally needs to operate to match the highest availability and security requirements of any of the services in which it participates, which makes it expensive and potentially slow for other services.

Another problem is assigning UUIDs to existing objects. The exclusion of legacy systems generally precludes adoption, and the provision for same generally requires extending the UUID much larger so that a UUID can be algorithmically derived from existing names, the whole external name in some cases.

However, the key, overwhelming, "you-heard-it-here-for-the-first-time" problem with UUIDs arises in trying to decide when an object should get a new UUID. That is, when an object is changed, at what point is it a different object? For example, if I completely rewrite a file, should it keep the UUID even though it has been completely changed? In one view, an object is *immutable* and the UUID identifies a given state of the object. Thus, a client holding a UUID can rely on the state to which it refers, and each modification to a file gives it a new UUID. However, that further complicates the directory mapping external names to UUIDs. In other cases, a client wants to see modifications to a given object, and thus does not want the UUID to change.

More specifically, consider the case of using UUIDs for files (i.e. they are the proper

[3]This specific embedding is ineffective unless the IP address is statically allocated, an increasing rare situation.

names for the files) and having the user rename a file, i.e. change its external name. Suppose the user renames "/A/B" to "/A/Bold", and then renames "/A/Bnew" to "/A/B". (This is standard behavior in replacing an old version of a program or library with a new installed version, when saving the old version.) How should UUIDs be handled in this case? There seems like no good answer. The user intention is really to have what was "/A/Bnew" replace what others were using previously as "/A/B". Yet, after the rename, "/A/Bold" really is "/A/B" so the UUID for what was "/A/B" should now refer to the file named as "/A/Bold". Things can also be confusing with copying files. What should be the UUID of an exact copy, if the UUID identifies content in some sense?

The core problem is that system-generated names, internal names, cannot capture user "intent" which is really the issue of whether an object has the same name or different.

These problems are avoided by not using persistent universally unique identifiers and relying external names. That is, make it a user decision by user control of external name space, i.e. application-specific semantics, and avoid UUIDs. Then, the meaning of a user-assigned name is whatever the user wants to make it mean, same content, different, etc. I.e. *user intent rules!*

5.3.4 Proper Use of Internal Names

External naming is required in any sensible system, and should be the "real" names for the objects, as advocated earlier. Thus, internal names are (at best) "just an optimization". However, in many cases they are a critical optimization. I.e. imagine routers switching packets based on character-string names in the packets!

Allow internal names to be short by making them unique only local to a context in which they are used. For example, an object identifier need only be unique relative to the communication channel to the node or server implementing the object. This handles the common case, namely for identifying the object as part of object operations. The name of the object can be used outside of this context to refer to the object. Because this is infrequent, the cost is not significant.

With this approach, the internal name can only be used between a client and the server, not between clients, but the former is the common case. References to an object passed directly between clients must use the object's external name. However, in many cases, the client only has the external name so this behavior should be supported in any case (otherwise, you force the client to get an internal name purely to pass a reference to another client.)

Allow internal names to be short by making them transient. When they are maintained only within the context of a transient connection such as a TCP connection, they are automatically transient. With more persistent "connections",

So, the question is: how big are these identifiers? Efficient internal names are short fixed-length bitstrings, i.e. 32 bits. (In the choice between 16, 32 and 64, there is frequently more than 65K instances of some object type, especially over time, but very rarely over 4 billion or even close, so 32 seems just credible. However, if you look at a maximum sensible lifetime to a connection and the maximum rate of identifier usage, it is often hard to argue that you stay under 4 billion objects during the lifetime of the connection. And, duplicate allocation in the rare case that you wrap is a nightmare. And, the overhead of checking for wrap, the mechanism for dealing with this and avoiding duplicate allocation is painful and rarely invoked. Thus, I think one ends up wanting 64 bits to avoid leaving little "time bombs" of potential wrap in the code. (it is feasible to have 4 billion objects, but not $2^{64} - 1$.) An alternative is to use a variable-length encoding, such as the varInt approach used by Google's protocol buffers mechanism. Then, in the common case, the identifiers are small but can grow as necessary to deal with the boundary cases.

Another approach is encoding a version field or similar in the identifier to facilitate controlling the reuse. You can look at a TCP connection as an object that implements this approach with its port number allocation plus the sequence number acting as a version number, as well as a packet sequencer. Conversely, one can recognize that a "connection" is really just synchronization of state to reliable delivery a sequence of packets, so one is really trying to both identify packets as well as sequence them. In this sense, the concatenation of IP addresses, port numbers of sequence number identifies a packet or actually data in the connection in TCP.

I have also looked at making internal names dependable by making them T-stable. That is, they do not change between valid meanings in less than some time T. Therefore, if the holder of the internal name checks its validity less than every T seconds, it detects the internal name becoming invalid, and thus a candidate to be rebound. That is, it is protected against the internal name changing meaning without it knowing. With this approach, the internal name just has to be big enough to accommodate $RT + S$ values, where R is max rate of creation/deletion and S is the most objects one expects to support at a time in this context. However, it seems simpler to just provide connection-based identifiers and ensure that an identifier never wraps.

Given the above approach to internal names, we need to provide a distributed external naming implementation that does not rely on UUIDs as proper names for objects. This is explored next.

5.4 External Naming Implementation

Character-string names chosen by user.

Requirements:

1) Stable storage and management of user-assigned names.

2) Prevention of ambiguity, i.e. duplicate names.

3) Uniform 'global' name space for all objects.

4) Efficient mapping and query — directory listing (for searching and exploring). Also, needs to efficiently determine if not found, i.e. not there.

5) Locality support (for efficiency and convenience): current working directory and abbreviations.

6) User customizability: aliases.

All objects into a large uniform and global name space, otherwise interoperability is a nightmare!

Thus, allow the whole world to be one naming domain, i.e., handle billions and billions of names.

Universality vs. Locality

- Global, location-transparent names are preferred, but long.
- High locality to use of names by users, both external and internal.

  E.g., most files accessed by user in his/her account directory. Thus, need for *abbreviations* for commonly used names.
- Context dependence - e.g. /dev/tty, ~/.cshrc

Aliases — support for user's personal preference by local renaming of an entity.

E.g., 'e' as alias for 'emacs' and emacs is an abbreviation for '/usr/local/bin/emacs'.

Need for multiple contexts for sharing, e.g., working directory of program invoker versus the working directory for program internal files.

Modular naming: module can be used without knowledge of the names it uses.

That is, no name conflicts or incorrect name resolution plus clean separation of user-dependent from user-independent binding.

Several problems: (1) lots of info, (2) heterogeneous (3) machine and comm. failures.

5.4.1 Centralized versus Decentralized Naming Authority

Centralized definition: one authority to consult on interpretation of name, name mapping. E.g., Internet name server — query for Internet address of specified host.

Decentralized definition: multiple authorities to consult. Interested party queries and picks from the multiple responses. E.g., to locate a file server, broadcast a query specifying the name and wait for the server to respond.

Search list mechanism: can be regarded as decentralized definition. Responses from different directories are prioritized by search order.

Extreme with centralized approach: one module defines all names.

Extreme with decentralized approach: every name definition is a separate module — each object is its own directory entry. However more practically, objects are implemented by object managers, and a single manager can provide a directory for all the objects it implements.

Centralized approach: more efficient in communication, no inconsistent responses but awkward to extend in distributed systems.

Decentralized approach: more expensive to communicate with all authorities and deal with multiple responses but easily distributed and robust.

The combined solution is the only scalable approach: cache info, structure name servers so one only has to access a single server in common case.

— resort to multiple servers on failure and other uncommon cases.

Returning to the implementation problem, efficient mapping of names required, plus support for binding and query (directory listing). The latter supports searching and generally understanding the substructure of a file organization.

Solution: multi-level global directory architecture.

5.4.2 Global Directory Architecture

Proposed directory architecture — not universally accepted.

Three levels (of course):

- global directory service — names *administrations*, human organization units, or groups of administrations.

  E.g., %edu as directory of educational institutions, and %edu/stanford as Stanford University. Also, DNS with naming of organizations such as ibm.com.

- administrational directory service — names directories of object and services etc. within an administration.

  E.g., hosts, users, services, mailboxes, etc.

- managerial directory service — names objects managed by individual managers or servers.

  E.g., files, windows, distribution lists, etc.

In the current world, the top level is handled by the DNS root name servers and the ISP-level name servers. The second level is handled by enterprise/campus-level DNS

and related servers. The bottom one is handled by the local file system, managed by the local administrator of that file server. In this sense, the different levels have different responsible organizations but may use some common technology.

Directories at different levels under different requirements, access characteristics.

Global Directories:

1. slowly changing, small amount of data.
2. of interest to all, so very high availability required.
3. entries from multiple different administrations so mistrust.

Administrational level:

1. single admin. or hierarchical trust.
2. High availability required within the administration, less so to outside world.
3. moderate rate of change, but change local to administration.

Managerial level:

1. rapidly changing, frequently accessed, more data.
2. availability depends on object being named, i.e., little need to make more reliable than object itself but possibly more than administrational directories.
3. security of naming depends on object being named,

Global directory service — implemented by dedicated, replicated name servers. e.g., Lampson design with full replication.

Managerial naming using decentralized approach as done in V.

— each manager covers its own objects.

Administrational directory service — directory servers that cache global and administrational directory information.

5.4.3 Lampson's Global Name Service

Ref: Designing a Global Name Service, 5th PODC, August, 1986.

Database, but (1) names change slowly, (2) integrity constraints weaker and (3) semantics/characteristics of the data can be exploited.

Requirements:

- large size.
- long lifetime.
- high availability.
- fault isolation.
- tolerance of mistrust.

Basic design: hierarchical directories, replicated for availability and performance.

Performance ⇒ 'read any, update all.'

Problem: atomic update of all is not feasible, due to crashes, expensive for large number of copies, e.g. long lock hold times.

Solution: Periodic propagation of updates to all copies using *sweeps*, rather than immediate, "pushing" for eventual consistency.

— read returns approximately up-to-date version, as of last sweep at least.

Sweep uses a 'ring' of directory copies to reliably identify all copies and a log of changes/updates.

Sweep:

1. Find all copies and updates 'next update time' to sweep time. (so new updates go to next sweep.)
2. Collect all updates.
3. Write all updates to all copies, updating last sweep time.

A sweep is somewhat like a transaction commit. However, the undo aspect makes it an asynchronous commit in some sense. Note: a sweep appears to need to resolve duplicate names in sweep operation.

Absence records represent deleted names, so this update can be propagated by the same mechanisms.

Logical timestamps order updates to naming database, imposing an order for resolution of possible conflicts. They also make the operations total, commutative and idempotent so reordering and redoing operations do not change the end state of the directory. (Note: real-time timestamps might be preferred now, given the availability of wide-area time synchronization, courtesy of NTP.)

Question: what happens if name is defined by two separate operations, starting in two separate copies?

One wins, the other is discarded (without notification?)

New epoch if ring needs to be reformed.

My guess: database resets to last completed sweep with a new epoch.

Other issues: (1) preventing cycles in name graph with back pointers, the back pointers are the primary or most believed information whereas the DR pointers are secondary.

— also helps find root from given directory.

(2) preventing circular dependencies in name lookup, using recursion where directory names map to server names.

Deficiencies:

1) Too inefficient for file naming, etc. (I dont think the design was intended to cover files, so one might rephrase the complaint "as it not being the single unified solution to naming".)

2) No support for locality.

3) No support for user customizability.

Versus these properties being provided by the decentralized V naming system, as described later.

5.4.4 Epidemic Approach

The Lampson approach is a highly structured approach to maintaining the directory service. Demers et al., Epidemic Algorithms for Replicated Database Maintenance, PODC'87. describe an unstructured approach in which they fancifully view an update as an infection. Mechanisms are provided to spread infection so that with high probability all are infected at low comm. cost and low time (average and last site) delay.

Three approaches:

- direct mail — updater directly contacts all.
- anti-entropy — a "simple epidemic" in which each node is susceptive or infective. In the simple case, there is a full exchange of state on "contact".

  Random pairs exchanging either by pull (ask if susceptible) or push (if infective).
- Rumor-mongering — susceptible, infective, removed (not spreading the rumor). The node maintains an update list and exchanges updates with other nodes. The problem is deciding when to remove an update from the list, and who to exchange with, such as finding "transit" nodes to distant collections of nodes.

Rumor mongering — efficient way to spread an update, but when does or should the rumor mongers lose interest in the rumor? E.g. when those they tell already know?

Issues of traffic required, delay and residual (those not receiving the update).

— what's the probability of missing someone?

Death certificates to propagate deletions, like immune system antibodies, that fight off old copies of deleted item. Note the common theme here relative to the Lampson system: deletion is represented as a thing denoting "gone but not forgotten."

Spatial distribution considerations — attractive to just contact the locals, but endangers the spreading the update. There seems to be considerable benefit in exploiting the typical hierarchy of the (inter) network topology in spreading infections.

Results: reduced link traffic by factor of average 4, and up to 30 on critical links with antialiasing and special distribution (vs. uniform distribution).

My assessment: a cute analogy but mapping the naming updates to the network topology which normally matches the naming hierarchy (roughly) which normally approximates the administrative hierarchy has too many benefits to ignore. Exploiting this structure provides an update propagation mechanism that eliminates most, if not all, of the "infection" and epidemic model, except possibly at a trivial level in detecting duplicates. I.e. not likely to be of much practical impact.

Possible exception: mostly disconnected operation.

More recently, the Dynamo system (DeCandia et al.) claims to use some aspect of "gossiping" (AKA rumor-mongering) for disseminating updates, but I have not fully understood the merits there.

5.4.5 Naming in the V-System

Approach in V: each server module implements its own directories of external names for the objects it implements. In this sense, it is a decentralized design.

Advantages:

- *efficiency* — eliminate going through name server on each access.
- *consistency* — external name and object in same module.
- *fewer levels of naming* — less need for global (internal) identifiers.
- *extensibility* — can handle established name spaces.
- *availability* — external name for object available if object is, because again in same module, i.e. fate sharing.

- *security* — name operations use same security mechanism as provided for accessing server/manager for other operations.

Issue: character-string name handling replicated in each server.

Note: uniform name-handling protocol for uniformity and 'server naming library' so only a RAM cost, and this cost is low, especially with modern shared library support.

I.e., Standardized operations for adding, deleting, modifying, etc. names.

This approach of providing directory associated with the module implementing the named objects takes advantage of the directory state semantics, namely providing mapping of name to object.

Note: inverse name mapping: what is global name corresponding to an object id? An object manager must remember name with which object was created.

Issue: how to efficiently locate the right module based on pathname? The answer is to cache name/path prefix information.

Caching

Name cache per client program — route to right server/directory before going over the network.

Prefix name cache — need only match on prefix of name.

Also, may match to directory identifier inside manager/server.

— returns (manager, local-directory-id)

Encoding of server/manager in directory id for efficiency.

Note: directory identifier is a transient, volatile identifier.

On-use detection of stale cache data.

Prefix matching allows very high hit ratios, e.g., 99.7 percent in V. (In fact, the primary source of misses were identified as typos.)

Range of 'hits' from subgroup of servers to individual server to subdirectory within a server.

Note: program pre-loaded on init to reduce startup cost from shell, however new names added to a programs cache are not added to shell's cache.

Directory identifier, allowing multiple 'current working directories' or contexts per server.

fully-qualified name — directory identifier and name relative to directory.

Run-time support maintains a *current directory* identifier plus prefix name cache.

Performance — direct unicast communication with server/manager unless cache miss (which is infrequent.)

> *Central name server would probably be remote, thus costing more.*

User Customizability Support

Alias names can be added to program cache, mapping to global name.

Cache translates an alias to its global name as part of the cache lookup.

Alias names are flagged so never dropped from the cache.

Aliases inherited in program execution mechanism by cache preload and local aliases discarded on program exit.

Directory Listing

Require a uniform way to query each object in a context.

Directory — logically a file of descriptions of every object whose longest proper prefix name is the name of this directory.

Each description is *typed* to indicate type of object and format of description.

Allows one 'list directory' program to work for all contexts, e.g., files, programs in execution, virtual terminals, contexts, etc. vs. Unix ls, ps, vmstat, etc.

Note: information need not be stored in server in form of directory, may be dynamically generated.

I.e., a 'view' (in database sense) of directory information data.

Alternative: enumeration service followed by individual queries.

Note: individual object query is standardized to match same description format.

Note: directory listing capability is an important service and one that is difficult to provide. The interface of providing a single entry at a time is inefficient. It is better to provide the ability to open a directory as a file and read. These leads to the notion of a adminstrative directory server as an optimization over a purely decentralized (multicast) directory implementation.

Overall, the V-system structure contrasts with the conventional distributed systems wisdom of the time, which was to:

1. give all objects UUIDs.
2. use (centralized) name server to map external names to internal names.

3. Name server is replicated for performance and reliability but logically one central service.

We have identified problems with this approach earlier.

The original V design used a totally decentralized naming.

No directory server — just multicast to locate names in servers.

Limitation: cannot distinguish 'undefined' from 'not available' or 'not reachable', i.e., network partitions.

Characterize as *binding check*: Is name N bound?

Basis for avoiding duplicate names.

Another issue: cost of multicasting to the world — high cost of typos.

One might summarize the V approach from the shared state standpoint, by saying: you co-locate the naming state with the object state for fate-sharing/availability, consistency and performance. (Performance benefits because you have to go to the server anyway to access the object.) In effect, the primary site for the object is the primary site for its meta information including its name.

5.4.6 Administrational Directories

The set of object managers implementing objects in one administrational directory implement that directory in a decentralized way.

Thus: communicate with this set of managers to map, query, modify an administrational directory.

Problems:

1) Incomplete and inadequate if a manager is inaccessible.

2) Expensive for repeated directory listings of admin. directories.

3) Remote access to local multicast groups for query, etc.

Solution: administrational directory servers.

Problem: Reliable persistent storage of directory information consistent with managers/servers.

Solution: store persistent (configuration) information in in global directory system and cache this information plus other information from manager/servers.

At min., global directories contains a list of names bound in the administrational directory.

Admin. directory server as a caching server — efficient access to admin. and global directories.

— interfaces local name protocol to global directory system(s).

Note: local administrational access continues even if all admin. directory servers fail. Thus, this level is just an optimization.

However, binding new names is dangerous and directory listings may be incomplete.

5.4.7 Security with Decentralized Naming

Problem: how does a multicasting client know that a responding manager is authorized to cover the queried name?

Solution 1:

1) Client knows the principals authorized for each portion of name space.

2) Client checks principal of responding manager.

Client must know name-principal mapping.

OK if simple and static, e.g., all file servers run as 'system'.

Solution 2: authenticator or capability from trusted 3rd party, e.g., authentication server.

Authenticator A: principal p(A) is authorized to cover names N(A) until time t(A), signed s(A).

Authenticator cannot be revoked but do expire.

Cost in performance and resiliency.

Overhead of authenticators in naming responses, verification and managers getting new authenticators after old expire.

Resiliency: servers cannot respond once authenticators expire if authentication service is inaccessible.

Trade-off: timeout on authenticator vs. performance/resiliency.

'Shadow' authentication hierarchy to naming hierarchy, possibly as a subtree of accounts, logically part of the naming hierarchy but implemented by the authentication server.

Solution 3: servers advertise their services periodically in terms of name space coverage. Clients only use servers that have been advertizing. If a rogue server is identified by it claiming to cover portions of the name space that is claimed by another server. I.e. you rely on the "social fabric" of the distributed system to avoid rogue servers. The advertising and lack of complaints builds that social fabric, i.e. reputation, AKA trust. In the limit, one can rely on DNS as a name service and then rely on the clients collectively recognizing and complaining about a DNS service when it goes bad. This structure follows our thinking on trust and observability, described in Chapter 1.

5.5 URLs vs. Academic Name Theory

Lampson's directory structure suggested that a name prefix maps to a directory server name. The directory server name is then used to contact the server to lookup the original name. The directory mapping name prefix to server name could also determine the protocol to use for this mapping. In theory, by locally caching name prefixes to servers, you can expect to see the mapping to server occur fairly efficiently.

In contrast, a URL explicitly specifies the protocol and server to handle the URL. For example,

```
http://www.stanford.edu/~cheriton/index.html
```

the first component specifies the access protocol, HTTP, and the second specifies the server explicitly, or at least the domain name for same. This structure conflicts with academic wisdom that embedding the host name into the object name is bad because it limits moving of the object without renaming it. Moreover, it contorts naming by forcing the name of the server (and protocol) into the object name. (I.e. it is an impure structured name with the server name as an impurity so you cannot move the object between hosts without changing its name.)

However, let's look hard at a URL. First, there is a user-visible and accepted notion of a *web site*, a unified collection of files/pages. A web site is more or less a basic unit of organization and migration, as called for in the first level of our naming/directory architecture. It also effectively represents the *brand* associated with the information, where "brand" connotes credibility, importance, etc. The "brand" associated with information becomes critical as systems scale. For example, `http://www.ibm.com/newproducts.html` has a very different brand or credibility than `http://www.bimbo.com/newproducts.html`. This relates to standard practice to ward off phishing attacks; you look hard at the URL for a "trusted" web site.

Second, a DNS name can represent a logical or virtual server, not necessarily a single physical server. For instance, "www.lastofthemohicans.com" maps to a server machine, but is the name of a web site, not a host. Thus, the web site can move between servers without changing the actual name of the servers, and without changing the name of the web site. With content routing, a DNS name can map to one of several different physically separate sites, not just separate server machines.

Third, web sites/pages can use redirect to support migration of subtrees that are below the unit of web site.

Moreover, removing the server name from the URL then requires a directory that maps name/URL prefixes to the server name. This directory needs to be global, a significant overhead in addition to the current global "host" naming. Moreover, the server directory needs to be global.

The primary case in which you run into trouble is splitting up a web site. For example, if Bill and Monica run "www.flowers.com" and Bill handles daffodils while Monica handles tulips. Just suppose they split up. In the Lampson world, you can imagine "//flowers/tulips" mapping to one server while "//flowers/daffodils" mapping to a second server after the split, with the global directory just being changed to recognize this split. With the current URL structure, it seems like one has to buy out the other from the use of "www.flowers.com", say Bill, and then forward access "www.flowers.com/tulips" to Monica's server. This is the same problem that businesses have in general in splitting up or breaking off a division, so it not a new problem. In reality of course, each company needs to build its own "brand", so needs its own name.

My conclusion is: there does not seem to be a really compelling reason to have the Lampson-style directory service, at least at the global level. In effect, the organization is visible in the name, and is usefully so, because it is important to the client. That is, scale means that the name needs a structure that indicates brand/trust, i.e. what is behind this object, a key part of its semantics. This contrasts with a digital signature-based scheme, which carries the implicit assumption of a trusted world-wide entity, a real impossibility.

Note that email addresses exhibit the same structure, I.e. cheriton@cs.stanford.EDU ties my email address to a particular organization. So, if I leave, my "name" changes. Some folks have their own domain names now, and simply forward to their current organization. However, a significant number of the attempts to contact me by that address are tied to my association with Stanford. In general, the organization is more important to be visible in many uses than not, given the organization confers some degree of legitimacy or relevance.

5.6 Location-based Naming and Identification

We defined *location-based identification* as identifying something by where it is physically. For example, you might say the "server in the Minneapolis point-of-presence" is down. Location-based naming is just using this form of identification in place of names.

At first thought, you might consider location-based identification a rather silly concept for "cyberspace" given it can be hard to determine where physically something is, and moreover, it generally irrelevant where something it as opposed to what it is or what principal on behalf it is acting.

However, as one scales a distributed system, there is an increasing risk of attacks, intruders and general participation of nodes that are necessarily trusted. Moreover, on examination of mechanisms that authenticate nodes, they are generally based

on *secrecy* in the form of keys for encrypting, signing, etc. Secrecy has terrible engineering properties:

1. The secret is generally a single point of failure. Once the intruder knows the (master) key, all the defenses are defeated.

2. There is no way to test or prove that some value is a secret. You simply have to take it on faith, presumably after taking some ad hoc measures that are supposed to favor it actually being a secret. For example, you may use a pseudo-random number generator to generate the next key yet the intruder may know enough about this PRNG to determine the next key, so it is not a secret even though it is *fresh*.

3. Relatedly, there is no way to determine when the system has failed. The intruder may have compromised the key and proceeded to monitor confidential information with no measurable change compared to an uncompromised system.

In contrast, location can support or undermine trust in actions in many situations. For example, if a company detects that the account of the CEO is being accessed from a wireless node and used to terminate the CFO, it is far more credible that this is a legitimate action if the accessing host is located in the CEO's office versus in the parking lot. Similarly, if a bank receives a request for a large transfer by a host that claims to represent Citibank, the request is more credible if the requesting server is located in New York than if the requesting server is located in Serbia. And, looking at the real-world experience in dealing with criminal activity (and warfare for that matter), location is important to determine who is doing what and how, and to hold them responsible.

Location is also often valuable as a bootstrapping mechanism to establish trust in identity. As exploited in KIWI (Faria and Cheriton), an enterprise corporate network can first decide whether to allow a wireness node access based on its location. If the node is located within the building, access is considered and else, not.

With these benefits, the question becomes the feasibility of locating a node reliably, especially with determined adversaries.

In KIWI, the idea is to provide a (higher) density of wireless access points (AP) that allows the system to accurately determine the location of a transmitter based on received signal strength at multiple locations. This determination is based on the physical property that signal strength attenuates roughly proportional to the square of the distance from the transmitter. Because multiple APs are receiving the same signal, the system can correct for different transmit signal strengths. The results indicate that one can, with high probability, locate a transmitter within 3-5 meters,

given a reasonable density of APs, such as one per 500 square feet (assuming some care in the environmental design to avoid excessive distortions and reflections)[4].

A further aspect is using this purported location for the system to determine a minimum signal strength to transmit to the node, including a network encryption key. If the intruder is not close to the determined position, it is unable to hear the transmission and thus unable to connect to the network.

In general, location-based security can thus be made to depend on physical properties, such as radio transmission, which the intruder cannot compromise. That is, there is no secret, just inherent properties of the universe.

It seems feasible to extend this approach to the wired Internet, especially with suitable support from the routing infrastructure. For example, if an intruder is required to use two-way communication to carry out an attack, the destination IP addresses need to map to an ISP and host where the intruder is located (or at least has an agent). An IP address can be mapped by its subnet to an ISP, and the ISP can, if cooperative, can provide further information on the location of the host. For example, if the access over a phone line to the ISP, it becomes feasible to trace back the endpoint of the phone line with the cooperation of the phone company.

Further extensions are possible. For example, routers could record aspects of traffic passing through them including input port, further supporting traceback. There has already been work on network-level traceback in support of responding to flooding denial-of-service attacks.

It is certainly possible to deploy networks and systems that make location-based identification difficult, if not infeasible. For example, a wireless network with low density of APs and complex radio environment may allow an intruder to compromise any location determination. Similarly, IP addresses are easily spoofed[5]. Also, with actions that do not require two-way communication, it can be infeasible to locate an entity. However, there is a long history of engineering environments and deployments to achieve properties such as "security" in the true sense of "safety". My view: we need to engineer networks so location can be determined and available as additional information to detect compromises as well as to provide a basis for establishing trust. Further, anticipating that no system is perfect and failures and compromises will arise, location as a means to identify who did what is crucial as part of recovery (back to state of trusted operation). This is the same as a community not being able to return to normalcy until a serial murderer is captured: Until you can identify the culprit, it's not clear who you can trust.

However, it appears likely that so-called "privacy advocates" will do their best to

[4]There are numerous caveats and complications based on potential use of directional antennas, multi-path effects, etc.

[5]I developed a shim protocol for IP called WRAP that provides path-based communication and thus path-based identification, which is essentially wide-area location-based identification.

prevent deployment of location-based identification until there is serious terrorist incident[6]. As technologists, our role is to try to push technically understanding of these issues so that the technology is known and understood when the deaths of thousands (if not millions) of people have caused society to wake up to the need.

5.7 Directory Services versus Databases

Naming system is a special case of a database; the name of an entity is just one attribute of the entity; name mappings are queries on (distributed) database.

X.500 — attribute-based naming structure.

Problems: efficiency, lack of distribution, complex interface for simple directory services.

Some have argued for the use general-purpose DBMS because:

1) growing directory functionality.

2) general-purpose overhead reduced by more, cheaper resources, e.g., CPU. memory and network bandwidth (and latency irrelevant?)

3) improvements in distributed database technology, including real-time databases, relaxed consistency, etc.

However, the reality is that scaling is critical for directory service, especially for the Internet, and general-purpose databases do not scale to sufficient degree (yet and perhaps ever). Thus, one can view a directory system as a specialized directory system, but also recognize that the specialization appears justified to meet scaling requirements.

5.7.1 UDDI

The *Universal Description, Discovery Integration(UDDI)* is an XML-based "registry" that was intended to allow businesses to register white page information (name and contact info), yellow page information (industrial categorization under standard taxonomies) and green page. Developed in 2000 originally, the vision was whole-wide registry by which consumers would link up with web service providers, as part of a broad service-oriented architecture trend. However, this effort seems to have stalled out, as indicated by the Universal Business Registry (UBI) work by SAP, IBM and Microsoft being discontinued in 2006.

UDDI appears to live on as a SOAP-accessed directory supporting the various XML-based SOA architectures in the enterprise space. As far as I know, SOAP is another

[6]One theory is that it is the lawyers trying to protect their unique privilege of "discovery", to dig into every aspect of your life, that are funding the so-called privacy advocates.

attempt to make RPC work, just burdened with the extra overhead of XML'ifying everything for extra overhead.

My theory is that the "universal" element of UDDI ran into the trust problem with scaling and the search clumsiness of directory systems that are primarily built for lookup. On the trust issue, how does one ensure any aspect of correctness to the UDDI entries. The analogy with the phone directory strikes me as rather bogus, because at least the phone companies require that you be a customer of theirs so they know your phone number and presumably billing address. A UBI entry could be just a random set of bits. Thus, UDDI is just used internal to enterprises where the trust is based on trust in the internal IT management.

On the search issue, searching in a directory of human-distilled "metadata" means the search is based on less information compared to a Google, and also means it is more subject to compromise. Finally, given the directory/registry is being provided as effectively a simple database for storing this metadata, the search mechanism is nowhere close to the capabilities of a dedicated search engine such as Google.

5.8 Resource Discovery and Search

There is a fallacy with the large-scale carefully thought-out designs described so far. It is very difficult to get these designs right the first time. By the time the technology and issues are well-understood, there is probably an system in place that is good enough. Moreover, design in the large tends to get compromised by politics, design-by-committee, legacy, etc. Thus, most very large-scale systems evolve by incremental development and extensions. For example, the Internet contains a plethora of half-baked, specialized protocols, such as DNS, RARP, ARP, ICMP, Finger, rwho, etc. Many of these protocols could be eliminated by providing the ideal, carefully designed global directory service as we have described so far. However, any such design would be highly compromised in the process of getting wide acceptance. For example, the X.500 grand design is clearly a product of a standards committee with lots of generality to cover a range of functionalities and implementations, as is prone to occur in committees. However, scant attention has been paid to making it implementable with the efficiency and simplicity necessary for wide-scale adoption. Designing the perfect directory service without such committee compromise is likely to be an academic exercise because it is unlikely to be adopted. Moreover, such grand designs tend to have many flaws until they have undergone considerable evolution, which brings us back to the earlier problem of evolutionary design.

An alternative approach is to build best-effort directory-like services, such as `netfind` (Schwartz91).

Schwartz and Tsirgotis (91) make several key points in supporting wide-area distributed information/directory services of this nature:

- Seed searches with lots of possibilities, e.g. the netfind seed database. Also, "pre-search" as done with search engines.
- Try multiple alternative sources of information, in parallel, for fault-tolerance. (This includes handling multiple protocols[7])
- Hang loose and simplify — tolerate differences in protocols implementation to the degree possible, and only use fancy features as performance optimizations. Do not compromise correctness.
- Retrieve information from where it resides "naturally", rather than introducing new information management burdens, i.e. do not store information unless you are prepared to maintain it.
- Handle more less clean information rather than depending on carefully editted and maintained directories. Rather like the "spell" model of spelling checking — filter the information to a short list for the user to review.
- Go for approximate solutions and discernible failure indications, so users do not expect perfect answers and can recognize when they do not get it.
- Recognize and accommodate multiple forms of hierarchies. Hierarchy is relevant for scalability, but there is no one ideal or agreed-upon hierarchical organization.
- Support flexible user-defined contexts for limiting searches and providing user-defined views of the information.
- Structure so that costs are reflected in the user interface, and (hidden) costs are bounded.

The wide-area network environment is basically a chaotic environment. One might argue for trying to build applications and systems that are intrinsically stable by strategies that effectively incorporate *attractors* into their dynamics so they are *self-organizing*.

Back in the early 1990's, I wrote: "With the enormous growth of the Internet and the lack of any real central authority, and the effective failure of ISO nightmare to become real, one should expect more dependence on techniques like netfind and other applications described by Schwartz in the future." In the late 90's, there was a goldrush for DNS name squatters and traders. There was also an explosion of content scamming to fool the basic search engines of the time. Then, along came Google.

[7] Rather like the world traveler, who needs to have a basic grasp at least of several languages.

Unlike the directory services such as X.500 and UDDI, Google has no dependence on some person's random notion of taxonomy or hierarchy or object model. In fact, to some degree it exposes a potential fallacy in lots of the work on large scale systems, namely that there is a single "model" that serves everyone.

Also, Google brilliantly deals with a key issue with directories and search, namely how to trust the content for relevance and indeed importance. The page rank approach effectively provides a reputation for every page relative to different search terms. This mechanism can be viewed as identifying importance or relevance as the key spectrum along which to gauge information where there is a lot of information. Here, incorrect or bogus information is just at the low end of relevance. Just eliminating bad information still leaves the overall issue of identifying the best information out of an ocean of information. Page rank largely solves the larger problem, thereby taking care of the trust problem as well. It is based on web-wide loose consensus by the ranking of pages by the links to those pages.

In general, search engines supported by *web crawlers* build up extensive databases from relatively unorganized information, significantly reducing the role of directories as means to find objects. Global naming is now largely a means to specify and identify within this search content, and from the search context in which a name is identified to its use. Search engines has also reduced the value of DNS names so short meaningful names are not so important. However, user control of the names and their mapping to content is still important.

5.9 Concluding Remarks

Directory services are a key functionality for distributed systems.

There is a plausible architecture for a global directory service. I have argued that the URL structure is an approximate realization of this global directory structure, and that there does not appear to be a compelling reason to improve on it.

However, a system-wide directory tends to be highly based on trust, which is either dependent on cryptographic (secrecy-based) techniques or on some advertising/social fabric techniques. This can be augmented by considering location-based identification, where identification is based on physical location, determined using physical properties. My theory is that anonymity and location-transparency in fullest sense are properties that we cannot afford on the large-scale (and are not particularly of concern on the small scale). I also claim that secrecy-based techniques (certificates, etc.) do not scale, so simply cannot be used/relied on.

The distributed directory approach was imagined by many to take on a more significant role, providing yellow page service, white pages, etc. And, specific generic names were assumed to be very valuable. However, search engine technology has

upended much of that direction. In the vast ocean of digital objects, the demand for determining "importance", the difficulty of doing this well, and the commercial value of companies such as Google that do do this well has limited directories to primarily mapping name to object. In my opinion, this is a good thing. However, scalable directory and location services could benefit from using some of the same social fabric techniques, i.e. advertising and reputation, rather than secret key-based approaches.

Readings

B. Lampson, Designing a Global Name Service, 5th PODC, August, 1986.

Demers et al., Epidemic Algorithms for Replicated Database Maintenance, PODC'87.

References

Dynamo: Amazons Highly Available Key-value Store G. DeCandia et al. SOSP, 2007

D. R. Cheriton and T.P. Mann, Decentralizing a Global Naming Service for Improved Performance and Fault Tolerance. ACM TOCS, May 1989

D. Faria and D. Cheriton, No Long-term Secrets: Location-based Security in Overprovisioned Wireless LANs, Proc. 3rd ACM Workshop on Hot Topics in Networks (HotNets-III), Nov. 2004.

M. Gritter and D. Cheriton, An Architecture for Content Routing Support in the Internet, 3rd Usenix Symp. on Internet Technologies and Systems (USITS), 2001.

M. Schwarz and P. Tsirigotis, Techniques for Supporting Wide Area Distributed Applications, U. of Colorado-Boulder Tech. report CU-CS-519-1, February, 1991

L. Peterson, The Profile Naming Service, ACM TOCS, 6, 4, Nov. 1988, p. 341-364.